Java Backend Interview Prep - Microservices

This document answers the Microservices topics listed in the source interview sheet.[file:1]

Architecture basics

1) Microservices vs monolith

A microservices architecture splits a system into independently deployable services aligned to business capabilities. A monolith packages most functionality into one deployable unit.[file:1]

Benefits:

- Independent deployment
- Team autonomy
- Targeted scaling
- Technology flexibility

Downsides:

- Distributed complexity
- More operational overhead
- Harder debugging
- Eventual consistency challenges

2) Service communication

Microservices communicate synchronously or asynchronously:[file:1]

- REST/HTTP
- gRPC
- Messaging via Kafka/RabbitMQ
- Rarely SOAP in modern systems

Rule of thumb:

- Use sync calls for immediate request-response.
- Use async events for decoupling and resilience.

3) Event-driven architecture

EDA uses events to notify other services that something happened. Producers publish events, consumers react asynchronously.[file:1]

4) Event vs command vs query

- **Event**: fact about something that happened.
- **Command**: request to do something.
- **Query**: request for data without changing state.

5) Message broker role

A broker decouples producers and consumers, buffers traffic, persists messages when needed, and supports retries or fan-out patterns.[file:1]

6) Event sourcing vs CRUD

CRUD stores current state directly. Event sourcing stores the sequence of domain events and reconstructs state by replaying them.[file:1]

Distributed consistency

7) Distributed transactions and Saga

Traditional ACID transactions across services are hard and expensive. Saga coordinates local transactions with compensation to achieve eventual consistency.[file:1]

Types:

- Choreography saga
- Orchestration saga

8) What happens if a saga step fails?

Completed prior steps are compensated if business rollback is needed. The system converges to a consistent state eventually, not instantly.[file:1]

9) Retries and idempotency in saga steps

Every step should be idempotent using request IDs, de-duplication keys, or state checks. Retries should use backoff and dead-letter handling.

10) High-throughput/global saga challenges

Main challenges:

- Duplicate messages
- Out-of-order events
- Cross-region latency
- Hot partitions
- Hard observability

Mitigations:

- Partition-aware event keys
- Idempotent consumers
- local transaction + outbox pattern
- tracing and monitoring

11) CQRS

CQRS separates write models from read models. Commands change state; queries read optimized views.[file:1]

Benefits:

- Better scaling of reads/writes independently
- Read models tailored to use cases
- Cleaner separation in complex domains

Trade-offs:

- More moving parts
- Eventual consistency
- Projection maintenance complexity

12) Keeping read model in sync

Common approach: publish domain events and update projections asynchronously. Outbox pattern is often used to reliably publish those events.

Resilience and networking

13) Feign client basics

Steps:

1. Add OpenFeign dependency.
2. Enable clients.
3. Define interface with mappings.
4. Externalize base URLs in config.
5. Add timeout, retry, and fallback behavior.

```
@FeignClient(name = "inventory-service", url = "${clients.inventory.url}")
public interface InventoryClient {
    @GetMapping("/api/items/{sku}")
    InventoryResponse getItem(@PathVariable String sku);
}
```

14) Fault tolerance vs resilience

Fault tolerance focuses on continuing despite failures. Resilience is broader; it includes adapting, degrading gracefully, recovering quickly, and protecting the rest of the system.[file:1]

15) Common resilience patterns

- Circuit breaker
- Retry
- Timeout
- Bulkhead
- Rate limiting
- Fallback
- Cache

16) Circuit breaker states

Typical states:

- Closed
- Open
- Half-open

17) Retry and exponential backoff

Retries help recover from transient failures. Exponential backoff prevents retry storms and reduces pressure on unhealthy downstream systems.[file:1]

18) Client-side vs server-side load balancing

- **Client-side**: caller chooses instance, often using service discovery.
- **Server-side**: load balancer/gateway chooses instance.

19) Kubernetes Service vs API Gateway

Kubernetes Service load balancing is usually internal service-to-service routing inside the cluster. API Gateway adds edge concerns like auth, routing, rate limiting, and aggregation. [file:1]

20) Layer 4 vs Layer 7 load balancing

- **Layer 4**: route based on TCP/UDP connection data.
- **Layer 7**: route based on HTTP semantics like path, host, headers.

For microservices, Layer 7 is often more feature-rich, but Layer 4 can be faster and simpler. [file:1]

21) Resilience4j circuit breaker example

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallback")
public InventoryResponse fetch(String sku) {
    return inventoryClient.getItem(sku);
}

public InventoryResponse fallback(String sku, Throwable t) {
    return new InventoryResponse(sku, 0, false);
}
```

22) Scaling

- **Vertical scaling**: add more CPU/RAM to one instance.
- **Horizontal scaling**: add more instances.

Microservices can scale particular hotspots independently, which is often a major advantage over monoliths.[file:1]

23) Kubernetes autoscaling

- **HPA**: scales pods based on metrics.
- **VPA**: adjusts pod resource requests.
- **Cluster Autoscaler**: adds/removes nodes.[file:1]

24) 12-factor apps and CAP

12-factor apps emphasize stateless processes, config in environment, disposability, logs as event streams, and parity across environments. CAP says a distributed system can only fully optimize two of consistency, availability, and partition tolerance at the same time.[file:1]

25) Service discovery and distributed tracing

Service discovery lets services find each other dynamically. Distributed tracing follows a request across services using trace IDs and span IDs for debugging latency and failures.[file:1]

In Spring Boot, tracing is usually implemented with Micrometer Tracing plus an exporter like Zipkin or OpenTelemetry-compatible backends.